

Implementing and Evaluating Atomic Multicast with Skeen's Protocol

Agatha Schneider

Pontifícia Universidade Católica do Rio Grande do Sul (PUCRS)

Porto Alegre, Brazil

htaschneider@icloud.com

Abstract—Atomic multicast is a fundamental communication primitive for distributed systems that require consistent ordering across overlapping groups of participants. [cite: 532] This paper presents the implementation and evaluation of a distributed key-value store based on Skeen's atomic multicast protocol together with an ACK-gated delivery variant proposed in the literature. [cite: 533] In addition to describing the implementation, we validate the protocol using deterministic correctness tests and evaluate its performance through reproducible benchmarks under both ideal and artificially delayed network conditions. [cite: 534] Our results quantify the cost of the additional synchronization step while illustrating the stronger ordering guarantees it provides. [cite: 535]

I. INTRODUCTION

Distributed applications frequently need to coordinate operations that span only a subset of the system while still preserving a consistent execution order. [cite: 536] Atomic multicast addresses this problem by guaranteeing that processes participating in the same multicast group observe messages in the same order. [cite: 537] Skeen's protocol is a classic solution to this problem and remains an important reference in the study of distributed systems. [cite: 538] While the original protocol provides efficient ordering, later work identifies executions in which stronger guarantees require an additional acknowledgement phase before delivery. [cite: 539] In this work, we present a practical implementation of both the original protocol and its ACK-gated variant as a configurable distributed key-value store. [cite: 540] Rather than focusing exclusively on the algorithm itself, we emphasize reproducibility: the project includes deterministic protocol validation, configurable multi-partition execution, automated benchmarking, and performance visualization. [cite: 541] The contributions of this paper are:

- A complete implementation of Skeen's atomic multicast protocol. [cite: 542]
- An implementation of the strengthened ACK-gated delivery variant.
- A deterministic testing framework capable of reproducing the ordering scenario discussed in the literature. [cite: 543]
- A reproducible benchmarking pipeline evaluating protocol overhead under varying cluster sizes and controlled network latency. [cite: 544]

A. Research Questions

Rather than proposing a new atomic multicast protocol, this work evaluates the practical implications of the ACK-gated

extension proposed in the literature through an independent implementation and experimental analysis. Our evaluation is guided by the following research questions:

- **RQ1.** Can the unsafe execution described for the original Skeen protocol be reproduced in an independent implementation?
- **RQ2.** Does the ACK-gated extension eliminate this execution under the same deterministic message schedule?
- **RQ3.** What performance overhead does the additional acknowledgement phase introduce under different cluster sizes and controlled network latency?

II. BACKGROUND

Atomic multicast is a group communication protocol that encapsulates the complex task of ordering requests across different partitions [cite: 44]. In partitioned distributed systems, this allows requests to be propagated consistently to specific sets of replicas without requiring every server in the system to process every request [cite: 446].

Skeen's algorithm is a classic approach to achieving this, where processes assign unique timestamps to multicast messages using logical clocks [cite: 234], [cite: 235]. However, the original protocol only guarantees "global total order," which is insufficient for applications requiring linearizability [cite: 174]. It fails to capture real-time dependencies between requests, meaning that partitions would need to implement additional ad hoc coordination to execute multi-partition requests correctly [cite: 174], [cite: 175].

To solve this, literature proposes an "atomic global order" property [cite: 186]. This strengthened property accounts for real-time relationships between messages by adding an extra condition: a message can only be delivered once every destination confirms it will not assign a lower timestamp [cite: 258]. This is implemented by adding an acknowledgement (ACK) phase [cite: 260].

III. IMPLEMENTATION

The system consists of a configurable distributed key-value store composed of independent partition processes communicating through a transport abstraction. Routing is configurable for an arbitrary number of partitions, allowing the same implementation to execute the original and strengthened protocols under different cluster configurations. The transport layer supports

both an in-memory implementation for deterministic evaluation and an HTTP-based implementation for distributed deployment.

The implementation also supports deployment using Docker containers for distributed execution. Performance experiments involving controlled network delay use a benchmark-specific latency transport that injects configurable delays into protocol messages, allowing network effects to be evaluated independently from protocol correctness.

We implemented a distributed key-value store to test both the original and strengthened Skeen’s protocols[cite: 540]. To simulate realistic network conditions, the system was deployed within Docker containers, allowing us to algorithmically generate network noise and latency.

In the original protocol implementation, when a client multicasts a message, each destination receives it, updates its logical clock, and replies with a local timestamp[cite: 240]. Once the sender collects all local timestamps, it calculates the maximum value to serve as the final timestamp[cite: 241]. Servers then deliver messages sequentially based on these final timestamps[cite: 243].

In our implementation of the strengthened variant, we extended this flow with the necessary final synchronization step. Once a server decides on the final timestamp for a message, it updates its clock and broadcasts an ACK to every other destination involved in the multicast[cite: 261]. The core difference is that a server is now strictly gated; it can only deliver the message after it has successfully received an ACK from every partition in the destination group[cite: 262].

A. Deterministic Protocol Validation

To validate the protocol independently of operating system scheduling and network timing, we developed a deterministic transport used exclusively during testing. Instead of immediately delivering protocol messages, the transport records them and allows tests to explicitly control the order in which they are released. This approach makes it possible to reproduce the execution discussed in the literature, verify the behaviour of both protocol variants under identical message schedules, and ensure that correctness results are fully reproducible.

IV. EXPERIMENTAL EVALUATION

A. Correctness Validation

The first stage of the evaluation focuses on correctness rather than performance. Using the deterministic transport, we reproduced the ordering scenario discussed in the literature for the original Skeen protocol and compared it against the ACK-gated implementation under the same sequence of protocol events. The original implementation exhibits the expected unsafe execution, whereas the strengthened protocol blocks premature delivery until acknowledgements have been received from every destination, producing an execution consistent with atomic global order.

B. Baseline Overhead

First, we measured the raw protocol overhead in memory, without artificial network delays[cite: 530]. Across a varying

number of destination partitions (from 1 to 5), the strengthened protocol consistently showed higher latency[cite: 530]. For instance, a multicast involving 3 destinations took between 484,907 and 489,826 nanoseconds per operation under the original protocol[cite: 530]. In contrast, the strengthened protocol took between 603,094 and 611,192 nanoseconds for the same 3 destinations[cite: 530]. This baseline increase is the direct computational and communication cost of the extra ACK phase.

C. Impact of Network Latency

Next, we algorithmically injected latency into the Docker network to observe how the protocols behave under less ideal conditions[cite: 531]. Because the strengthened protocol requires an extra round trip for the ACKs, we expected it to be more sensitive to delays.

When injecting a 5ms delay on a 3-destination multicast, the original protocol required approximately 39.3 to 39.4 million nanoseconds per operation[cite: 531]. Under the exact same 5ms delay, the strengthened protocol’s completion time jumped to over 61.6 million nanoseconds[cite: 531]. This pattern held true across the board; as latency increased to 10ms, the performance gap widened further, directly reflecting the cost of waiting for global ACKs over a slow network[cite: 531].

V. DISCUSSION

Our implementation and subsequent benchmarks clearly illustrate the trade-off between strict ordering guarantees and system performance. The original Skeen’s protocol is faster and more resilient to network latency because it requires fewer communication steps[cite: 248]. However, to achieve linearizability—a strict requirement for modern, strongly consistent databases—the original protocol forces the application layer to coordinate during the actual execution of the request [cite: 175],[cite: 182].

The strengthened variant effectively pushes this coordination down into the multicast layer[cite: 183]. By ensuring atomic global order natively, partitions do not need to pause and signal each other before executing a delivered message[cite: 453]. While this drastically simplifies the application logic, our latency benchmarks prove that this extra communication round makes the system highly vulnerable to network delays[cite: 531].

A. Limitations

The current implementation models one process per partition and assumes static cluster membership. It does not include replicated partition groups, crash recovery, persistent storage, or dynamic reconfiguration. Consequently, the evaluation isolates the behaviour and performance of the multicast protocol itself rather than the operational characteristics of a production-ready distributed database.

VI. CONCLUSION

In this paper, we built a fully functional distributed key-value store to evaluate Skeen’s original atomic multicast protocol

alongside its strengthened, ACK-gated variant [cite: 540],[cite: 542]. Through deterministic correctness tests and reproducible benchmark scenarios, we verified that our implementation exhibits the expected behaviour described in the literature while quantifying the practical cost of the additional acknowledgement phase.

Our benchmarks demonstrate that while the ACK phase guarantees linearizable request ordering without application-level coordination, it introduces a measurable performance penalty [cite: 534],[cite: 535]. This overhead is manageable in fast, local networks but degrades performance significantly as network latency increases, highlighting a fundamental trade-off in distributed system design.